

Tape Drive Devices

Tape devices are character devices on a Linux system, going by a variety of filenames:

- SCSI tapes use the names `/dev/st0`, `/dev/nst0`, `/dev/st1`, `/dev/nst1`, and so on. The SCSI tape drive interface and driver is widely regarded as the most reliable, but, of course, SCSI tape drives are also more expensive than others.
- ATAPI tape devices start at `/dev/ht0` and `/dev/nht0`.
- There is limited support for tape drives on the floppy controller at `/dev/ft0` and `/dev/ntf0`.

To identify a tape drive, look in your kernel messages for lines like this:

```
Vendor: HP          Model: C1533A          Rev: 9503
Type:   Sequential-Access      ANSI SCSI revision: 02
st: Version 20010812, bufsize 32768, wrt 30720, max init. bufs 4, s/g segs 16
Attached scsi tape st0 at scsi0, channel 0, id 4, lun 0
```

There are two versions of each tape device on a Unix system:

- A *rewind* tape device rewinds the tape after every operation. For example, `/dev/st0` is a rewind device.
- A *no-rewind* tape device does not rewind the tape after an operation. No-rewind devices start with `n`; for example, `/dev/nst0` is a no-rewind device.

For day-to-day operation, rewind devices are practically useless. You may as well forget that they exist; the next few sections show that you often need to reposition the tape.

When working with tapes, you sometimes need to know the tape block size. Applications tend to write to the tape in a certain block size, and many tape drives do not allow you to read from the tape if you specify the incorrect block size. This is almost never a problem, because you normally write to and read from the tape with the same program. However, if you want to use `dd` to get direct access to tape files, you may need to specify the block size manually (see [Direct File Access](#)).

Before you get started with a tape drive, you should set the `TAPE` environment variable to the no-rewind device name. This variable saves a lot of typing in the long run, in particular, with `-f` options for the `mt` and `tar` commands (which are explained in the next section).

Working with Tape Drives

A tape is one of the simplest forms of media available. You can think of a tape as a sequence of files with a *file mark* between each file, as shown in [Figure 13-1](#). Notice how there is no file mark at the beginning of the tape and that the first file is numbered 0.

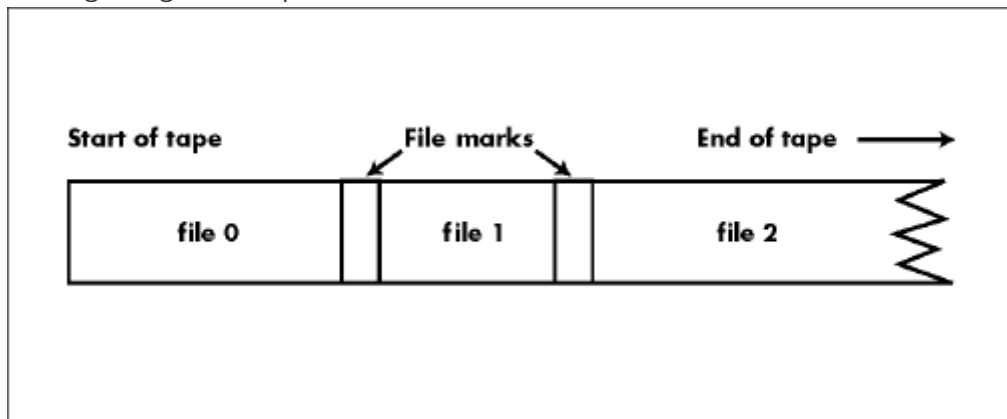


Figure 13-1. Files and file marks on a tape.

A tape does not contain a filesystem. Furthermore, there is no information on a tape indicating the name of each file (though automated backup systems usually include a special tape index file at the beginning of the tape). When manually working with a tape, you work with file numbers and file marks, and nothing else.

To access the data on a tape, you must manipulate the tape drive head. [Figure 13-2](#) shows the initial position of the tape head. Let's say that your tape device is `st0`, so you're using the tape device `/dev/nst0`. First, verify the tape position with the `mt status` command (the `-f file` option specifies the tape device file; remember that you do not need this if you set the `TAPE` environment variable):

```
mt -f /dev/nst0 status
```

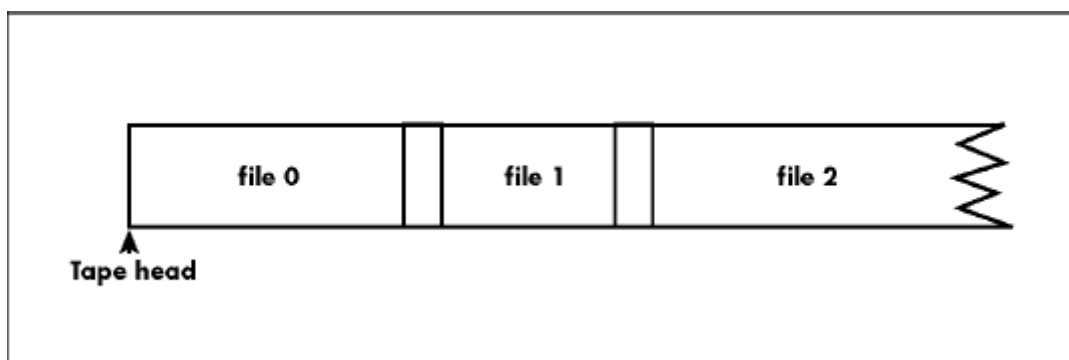


Figure 13-2. The tape head at the beginning of a tape.

The output should look like this (the following output is from a DAT drive):

```
SCSI 2 tape drive:
File number=0, block number=0, partition=0.
Tape block size 0 bytes. Density code 0x13 (DDS (61000 bpi)).
Soft error count since last status=0
General status bits on (41010000):
BOT ONLINE IM_REP_EN
```

There's a lot of information here:

- The current file number is 0.
- The current block number is 0, so the tape head is at the beginning of a file.
- You should ignore the partition; most tape drives don't support it anyway.
- The block size is 0, meaning that the tape drive does not have a fixed block size.
- The density code indicates how much data can fit on the tape.
- The soft error count is the number of recoverable errors that have occurred since the last time you ran `mt status`.
- General status bits explain more about the state of the tape and tape drive. `BOT` means "beginning of tape," and `ONLINE` reports that the tape drive is ready and loaded. See [mt Commands and Status](#) for a full list of `mt` status bits.

Creating Archives on a Tape

It is difficult to understand how a tape drive works by running through various files on a random tape in a collection of backups. Therefore, to explain the process of searching through a tape, this section shows you how to create some archives for yourself before going into the specifics of unarchiving. Once you have learned this, you will be able to play with a tape without significant confusion or worrying about losing data.

Here is a procedure for creating three files on a tape — specifically, `tar` archives of `/lib`, `/boot`, and `/dev`:

1. Put a new tape in your drive (a blank tape, or one that you don't care about).
2. Make sure that your `TAPE` environment variable is set to the no-rewind tape device.
3. Run `mt status` to verify that the tape is in the drive and your `TAPE` variable is set.
4. Change to the root directory.
5. Create a `tar` archive of the first file on the tape with this command:

```
tar zcv lib
```

6. Run `mt status`. The output should look something like the following, indicating that the tape is on the next file (file 1):

```
SCSI 2 tape drive:
File number=1, block number=0, partition=0.
Tape block size 0 bytes. Density code 0x13 (DDS (61000 bpi)).
Soft error count since last status=0
General status bits on (81010000):
EOF ONLINE IM_REP_EN
```

7. Create an archive of your `/boot` directory:

```
tar zcv boot
```

8. Create an archive of your `/dev` directory:

```
tar zcv dev
```

The contents of your tape should now look like [Figure 13-3](#).

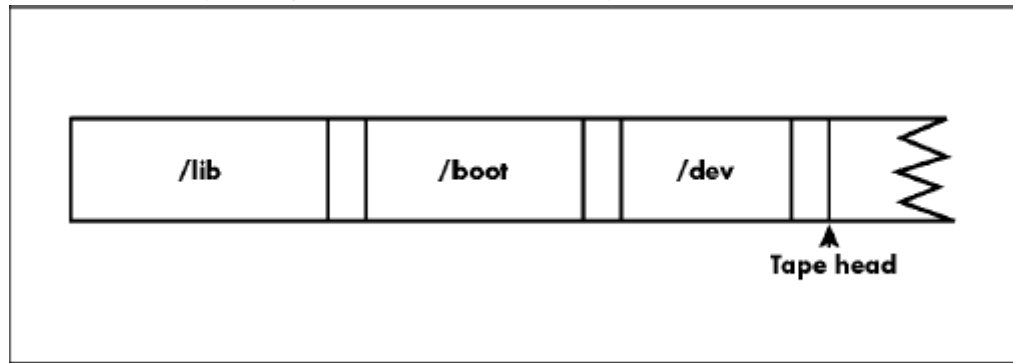


Figure 13-3. The tape containing three archive files with the head at the start of a new file.

Reading from Tape

Now that you have some files on the tape, you can try reading the archive:

1. Change to the `/tmp` directory to avoid any accidents (in case you accidentally type `x` instead of `t` in one of the steps that follow).
2. Rewind the tape:

```
mt rewind
```

3. Verify the first file on the tape, the archive of `/lib`:

```
tar ztv
```

A long listing of the files in the archive should scroll by.

4. Run `mt status`. The output should look like this:

```
SCSI 2 tape drive:
File number=0, block number=4557, partition=0.
Tape block size 0 bytes. Density code 0x13 (DDS (61000 bpi)).
Soft error count since last status=0
General status bits on (1010000):
  ONLINE IM_REP_EN
```

The tape is still at file 0, the first file on the tape. How did this happen? The answer is that as soon as it finishes reading, `tar` stops where it is on the tape, because you may want to do something crazy (such as append to the archive). The tape head is now at the very end of file 0 (notice the block count in the `mt` output); see [Figure 13-4](#).

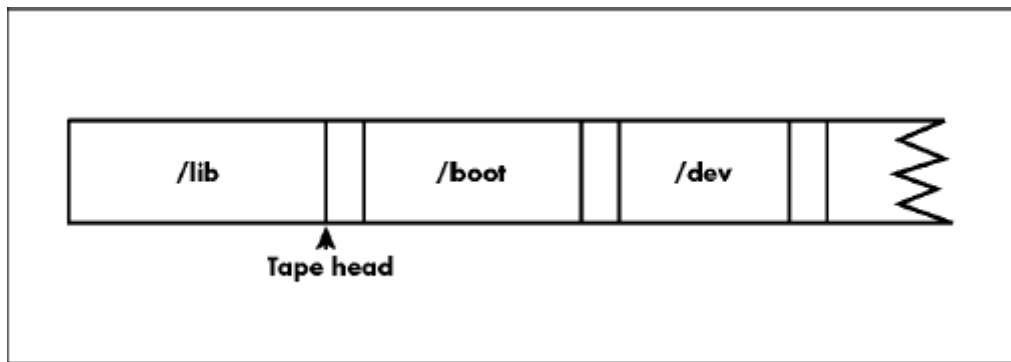


Figure 13-4. The tape head at the end of file 0.

The most important consequence of the new tape position is that another `tar ztv` does not access the next archive on the tape, because the head is not at the beginning of the next file, but rather, it is at the beginning of the file mark. To advance the tape forward to the next file (the `/boot` archive), use another `mt` command:

```
mt fsf
```

This command moves the tape so that the head is at the end of the file mark (the beginning of the `/boot` archive), as shown in [Figure 13-5](#).

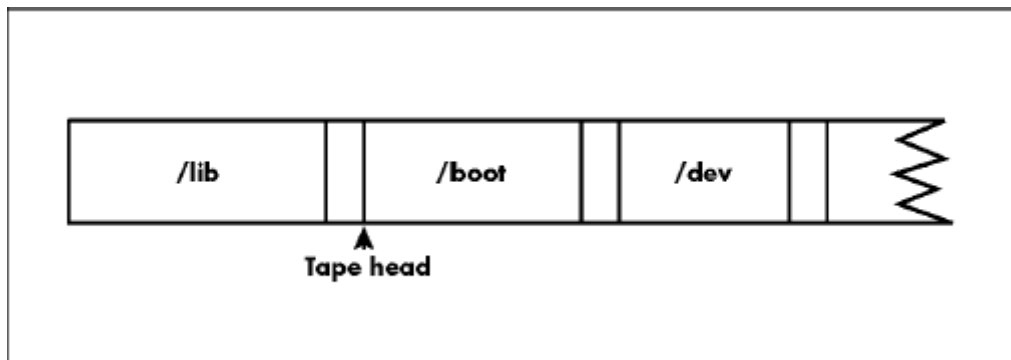


Figure 13-5. The tape head at the beginning of file 1.

Extracting Archives

To extract archives from a tape, all you need to do is replace the `t` in the commands in the previous section with `xp`:

```
tar xpv
```

As usual, you should double-check your current working directory before extracting any archives.

Moving Forward and Backing Up

The `mt fsf` command moves the tape to the start of the next file. There is also a command for moving the tape back:

```
mt bsf
```

However, it is important to understand that this command rewinds the tape to the end of the previous file on the tape; that is, to the start of the next file mark that the tape head sees.

Let's say that you are at the start of file 1, as shown earlier in [Figure 13-5](#). Running `mt bsf` reverses the state to that shown in [Figure 13-4](#). Try it for yourself, and then run another `mt status`. The file number should be 0 with a block number of -1 (the driver cannot compute the length of the file by itself).

In general, to get back to the start of the previous file when you are at the start of a file, you need to move back twice and then forward once. Because most `mt` tape movement commands take an optional file count (an example is the 2 here), you can run the following sequence:

```
mt bsf 2
mt fsf
```

However, there's another gotcha here, because the tape in this example is at the start of file 1. Running `mt bsf 2` attempts to move the tape back two file marks. However, the beginning of the tape is not a file mark. Therefore, the `mt bsf 2` in this sequence backs up, passing one file mark, and then hits the beginning of the tape, producing an I/O error. The subsequent `mt fsf` skips forward again, moving the tape back to the beginning of file 1.

Therefore, if you want to get to file 0, you should always rewind the tape instead with `mt rewind`.

Note The Linux `mt` includes a `bsfm` command that is supposed to take you directly to the start of the previous file. This nonstandard extension does not work quite correctly; if the tape head is at the start of a file, `mt bsfm` runs `mt bsf`, then `mt fsf`, taking you right back to where you started.

The bottom line is that you need to pay careful attention to the tape head position with respect to the file marks and choose the appropriate command. If it all seems a little ridiculous to you, it's probably not worth sweating over; to get to file `n` on a tape, this always works (though it may be slower):

```
mt rewind
mt fsf n
```

To eject the tape, use the `offline` command:

```
mt offline
```

mt Commands and Status

These are the most common `mt` commands:

- `mt status`
Produces a status report.
- `mt fsf n`
Winds forward to the start of the first block of the next file. The `n` parameter is optional; if specified, `mt` skips ahead `n` files instead of just one.
- `mt bsf n`

Winds back to the end of the previous file. The `n` is an optional file count.

- `mt rewind`

Rewinds the tape.

- `mt offline`

Rewinds the tape, then prepares for tape removal. In some tape drives, the tape drive ejects the tape; in others, the drive unlocks the tape so that you can manually remove it.

- `mt retension`

Winds the tape all the way forward and then rewinds. This sometimes helps with tapes that you're having trouble reading.

- `mt erase`

Erases the entire tape. This usually takes a long time.

There are countless other `mt` commands documented in the `mt(1)` manual page, but they really aren't very useful unless you're trying to extract blocks from the center of a file or to set SCSI tape driver options.

Running `mt status` yields a number of status bit codes (such as the BOT code from [Working with Tape Drives](#)). Here are the important codes:

- `ONLINE`

The drive is ready with a loaded tape.

- `DR_OPEN`

The drive is empty (possibly with the door open).

- `BOT`

The current position is the beginning of the tape.

- `EOF`

The current position is the beginning of a file (the end of a file mark). This is a somewhat misleading code, because you can confuse it with the end of file.

- `EOT`

The current position is the end of the tape.

- `EOD`

The current position is the end of recorded media. You can reach this by trying to `mt fsf` past the very last file marker.

- `WR_PROT`

The current tape is read-only.

Direct File Access

`tar` isn't the only command that can write archives to a tape. Sometimes you may not know what is on the tape, and even if you do know, you may want to copy a file from the tape to the local filesystem for further (and easier) examination.

Your good old friend `dd` can do this, as this example for `/dev/nst0` shows:

```
dd if=/dev/nst0 of=output_file
```

This doesn't always work — you may get an I/O error because you did not specify the correct block size. The program that writes the archive on the tape usually determines the block size; in the case of `tar`, the block size is 10KB (specified to `dd` as `10k`). Therefore, because `dd` defaults to a block size of 512 bytes, you would use the following command to copy a `tar` archive from the tape to `output_file`:

```
dd bs=10k if=/dev/nst0 of=output_file
```

Block sizes for common programs are listed in [Table 13-1](#) (see also [Other Archivers](#)).

Table 13-1. Block Sizes for Common Archiving Programs

Archiver	Block Size	bs Parameter
<code>tar</code>	10KB (20 × 512 bytes)	<code>bs=10k</code>
<code>dd</code>	512 bytes	<code>bs=512</code>
<code>cpio</code>	512 bytes	<code>bs=512</code>
<code>dump</code>	10KB (20 × 512 bytes)	<code>bs=10k</code>
Amanda	32KB	<code>bs=32k</code>

Note Unlike `tar`, `dd` advances the tape head to the next file on the tape, instead of just moving it up to the file mark.